

LARA Planning-Separation Ablation: Does Splitting Explorer from Executor Help?

Same-day, paired comparison of full LARA (Explorer + Executor) against a single agent holding identical specialist knowledge and a larger step budget, gpt-4.1-mini, 45-task slice

Bottom line

Removing the separate planning stage — while transplanting the Explorer's full discovery-and-planning knowledge directly into a single agent's prompt, and giving that agent nearly double the step budget to compensate for doing its own API discovery inline — cost **31.1 TGC points** (73.3 → 42.2) and is **statistically significant** (McNemar exact $p = 0.0043$, 22 discordant pairs). Both arms ran the same day on the same slice, closing the gap that undermined an earlier version of this comparison (see "Why this was re-run," below). Separating planning from execution is a real contributor to LARA's accuracy, not an artifact of comparing runs from different sessions.

What was ablated

Two arms, same pipeline, one architectural split removed:

Arm	Description	Isolates
A — full (shipped)	Explorer writes an [app]-tagged plan; Executor runs a ReAct loop against it. 16 ReAct steps.	reference
B — single-agent (v2)	No separate Explorer node. One agent gets the Executor's ReAct loop <i>plus</i> the Explorer's full planning/discovery system prompt, transplanted live via <code>build_explorer_system()</code> so it cannot drift from the real Explorer's text. 30 ReAct steps.	whether a <i>separate planning stage</i> matters, independent of whether the knowledge to plan is available

Arm B keeps every specialist prompt and the full cross-app ledger — identical to arm A. The only structural difference is whether planning happens in its own LLM call before execution begins, or is folded into the same agent that then executes.

Why 30 steps, not 16, for arm B

In arm A, the Explorer's discovery work happens in separate LLM calls that never touch `MAX_REACT_STEPS`, so the Executor's 16 steps are 16 steps of pure execution. Arm B has to do that same discovery *inside* its ReAct loop, charged against the same step count. An earlier equal-budget run showed several tasks exhausting all 16 steps without finishing — a budget-exhaustion artifact, not evidence about planning per se. 30 steps gives arm B roughly the same amount of real execution budget as arm A gets for free. **This is a deliberate handicap in arm A's favor removed**, not evidence stacked in favor of the hypothesis — if anything it makes arm B's underperformance harder to attribute to running out of steps.

Step-ordering mitigation

Arm B's initial plan is a stub ordered by a keyword table, not by task logic, so it can start a task dispatched to the wrong specialist. To reduce this confound, arm B re-parses the agent's own first self-authored `PLAN`: output (if any) and rebuilds the step→specialist dispatch map from it — see `rebuild_map_from_output` in `app_agents/base.py`. This engaged on **28 of 45 tasks** in this run. It is a mitigation, not a full fix: on the other ~17 tasks the agent never printed a re-parseable

plan, so those tasks may still carry some routing-order penalty unrelated to the planning-separation question itself (see Caveats).

Method

Both arms ran the identical fixed 45-task slice (`extended_slice.json`, train only), in identical task order, on `gpt-4.1-mini` via the same API gateway, **on the same day** (2026-09-06). Paired design: every task appears in both arms.

Held constant: `MAX_EXECUTOR_RUNS = 1` (no retry), `ENABLE_REVIEWER_RETRY = False`, all 10 specialist prompts, the cross-app ledger, `BOOTSTRAP_CODE`. Only whether planning is a separate stage, and the resulting step budget, differ.

Token counts are the provider's own usage figures, recorded per call (`token_meter.py`), not estimates.

Results

arm	TGC	SGC	d1	d2	d3	tokens/task
A — full	73.3 (33/45)	64.0	92.3	75.0	56.2	72,490
B — single-agent	42.2 (19/45)	40.0	46.2	37.5	43.8	76,560

Paired McNemar (exact, two-sided): 18 tasks solved only by A, 4 only by B, 15 by both, 8 by neither — **$p = 0.0043$, significant**. 22 discordant pairs comfortably clears the "fewer than 10 = directional only" threshold that limited other ablations in this project — this is a real, resolvable effect at $n=45$.

Token cost is nearly identical between arms (72,490 vs 76,560, a 5.6% difference) — arm A is simultaneously **cheaper and far more accurate**. This is not a case of one arm buying accuracy with extra spend.

Difficulty-1 shows the largest absolute gap (92.3 vs 46.2), not difficulty-3 as originally hypothesized in earlier versions of this ablation — see below.

Why this was re-run

An earlier version of this comparison used a full-LARA baseline scored on 2026-08-19 (TGC 64.4) against a single-agent arm scored the same day (TGC 31.1), producing a $2.1\times$ ratio and a 33.3-point gap. Separately, the specialist-dispatch study (2026-09-04) found that re-running an unchanged full-LARA configuration on a different day produced meaningfully different scores ($64.4 \rightarrow 75.6 \rightarrow 71.1$ across three separate sessions on the same slice with zero code changes) — evidence of substantial run-to-run variance on `gpt-4.1-mini`, though **not evidence that this variance is specifically calendar-driven rather than ordinary stochastic noise between any two runs**. Both explanations are consistent with the data; nothing here distinguishes them, and this report does not claim otherwise.

Given that uncertainty, the only defensible way to re-test the planning-separation claim was to run both arms close together, so that whatever variance exists is shared as much as possible rather than attributed to one arm. Today's arms landed at 73.3 and 42.2 — higher than August's 64.4/31.1 in both cases, consistent with generic run-to-run variance affecting both arms similarly, while the **gap between them held** (31.1 pts today vs. 33.3 pts in August). This is the reassuring outcome: the architecture's contribution appears robust to whatever is producing the day-to-day score movement, because that movement seems to move both arms together rather than favoring one.

Caveats

- **Step-ordering confound only partially addressed.** The self-authored-plan rebuild engaged on 28/45 tasks; the remaining ~17 may still carry some routing-order penalty from the keyword-table stub that is unrelated to planning-separation per se. This could inflate arm B's failure count somewhat. The direction of the effect ($A > B$) is unlikely to be an artifact of this alone given the size of the gap, but the exact magnitude should be read with this in mind.
- **Two runs, not a distribution.** No within-day repeats of either arm exist here, so there is no direct estimate of same-day variance for *this* pair — only the indirect evidence from the specialist-dispatch study that full-LARA alone can vary by ~10 points across sessions. If arm A's "true" score is somewhere in the 64-76 range seen across three sessions, the specific ratio (1.74×) should be treated as approximate; the direction and rough magnitude of the gap are what this run supports most confidently.
- **Single model tier.** gpt-4.1-mini only. Whether a separate planning stage matters as much for stronger models (e.g. claude-opus-4-7) is untested here — the model-tier study referenced elsewhere in this project's docs suggests architecture contributions shrink, but do not vanish, at stronger tiers for *specialist* knowledge; whether the same holds for *planning separation* specifically has not been measured.
- **Deliberate step-budget asymmetry (16 vs 30) remains uncontrolled for in the other direction** — i.e., this study does not test whether giving arm A a *larger* budget than 16 would close any of the gap. That is a distinct question (see RESULTS_SUMMARY.md for other suggested follow-ups) and was not run here.
- Train split only; held-out test splits were not touched.